

STRATEGY. Read all problems first (1 min). Order by speed-of-points. Bank easy points fast.  
 Stuck rule: 8–10 min no traction → move on, come back. Talk out loud — state approach in one sentence.  
 For SQL: write output columns + grain on paper first. Mention edge cases: empty, ties, NULLs, dupes, off-by-one.

## SQL — WINDOW FUNCTIONS

### Ranking within group

```
SELECT *,
    ROW_NUMBER() OVER (PARTITION BY dept
        ORDER BY sal DESC) AS rn
FROM emp;
-- ROW_NUMBER 1,2,3,4    RANK 1,2,2,4
-- DENSE_RANK 1,2,2,3
```

### Top-N per group

```
WITH r AS (
    SELECT *, DENSE_RANK() OVER (
        PARTITION BY dept ORDER BY sal DESC
    ) AS rk FROM emp
)
SELECT * FROM r WHERE rk <= 3;
```

### Running total

```
SUM(amt) OVER (
    PARTITION BY user_id ORDER BY dt
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND CURRENT ROW)
```

### Prev / next · deltas

```
LAG(amt, 1) OVER (PARTITION BY user_id
    ORDER BY dt) AS prev_amt
LEAD(amt, 1) OVER (...) AS next_amt

amt - LAG(amt) OVER (...) AS delta
```

### Buckets · moving average

```
NTILE(4) OVER (ORDER BY rev DESC)
    AS quartile

AVG(x) OVER (ORDER BY dt
    ROWS BETWEEN 6 PRECEDING
    AND CURRENT ROW) -- 7-day MA
```

### Common templates

```
-- Nth highest salary
SELECT DISTINCT sal FROM emp
ORDER BY sal DESC LIMIT 1 OFFSET 1;

-- Consecutive duplicate rows
SELECT id, val FROM (
    SELECT *, LAG(val) OVER (ORDER BY id)
        AS prev FROM t
) x WHERE val = prev;

-- In X but not Y
SELECT user_id FROM o WHERE prod='X'
EXCEPT
SELECT user_id FROM o WHERE prod='Y';
```

## SQL — AGGREGATION PATTERNS

### Conditional aggregation (pivot via CASE)

```
SELECT user_id,
    SUM(CASE WHEN status='paid'
        THEN amt ELSE 0 END) AS paid,
    SUM(CASE WHEN status='refund'
        THEN amt ELSE 0 END) AS refund,
    COUNT(CASE WHEN status='paid'
        THEN 1 END) AS paid_n
FROM txn GROUP BY user_id;
```

### WHERE vs HAVING

```
SELECT dept, AVG(sal) FROM emp
WHERE active = 1 -- before group
GROUP BY dept
HAVING AVG(sal) > 100000; -- after
```

### First / last per group

```
SELECT user_id,
    MIN(dt) AS first_seen,
    MAX(dt) AS last_seen,
    COUNT(DISTINCT session_id) AS sess
FROM events GROUP BY user_id;
```

### Joins

```
-- Anti-join (in A, not in B)
SELECT a.* FROM a LEFT JOIN b
    ON a.id = b.a_id
WHERE b.a_id IS NULL;

-- Self-join (employee -> manager)
SELECT e.name, m.name AS mgr
FROM emp e LEFT JOIN emp m
    ON e.mgr_id = m.id;
```

### NULL handling

```
COALESCE(col, 0) -- replace NULL
NULLIF(a, b) -- safe divide:
    a / NULLIF(b, 0)
col IS NULL (never col = NULL)
COUNT(*) counts rows; COUNT(col) skips
```

### Dates

```
-- MySQL
DATEDIFF(d2, d1)
DATE_ADD(d, INTERVAL 7 DAY)
DATE_FORMAT(dt, '%Y-%m')
-- Postgres
d2 - d1 (days, int)
DATE_TRUNC('month', dt)
EXTRACT(YEAR FROM dt)
```

### SQL Gotchas

- LEFT JOIN ... WHERE b.col=x silently becomes INNER. Put filter in ON.
- NULL = NULL is NULL (false). Use IS NULL.
- Integer division — cast: 1.0\*a/b or CAST(a AS FLOAT)/b.
- Join cardinality: dupes on right multiply left rows. COUNT(\*) before/after.

import pandas as pd · import numpy as np · from collections import Counter, defaultdict

## GROUPBY & AGGREGATE

```
df.groupby('user_id')['amt'].sum()

df.groupby('user_id').agg(
    total=('amt', 'sum'),
    n     =('amt', 'count'),
    avg  =('amt', 'mean'),
)

df.groupby(['user_id', 'month']).agg(...)

# Top-N per group
(df.sort_values(['dept', 'sal'],
    ascending=[True, False])
 .groupby('dept').head(3))
```

## MERGE (= SQL JOIN)

```
df.merge(other, on='id', how='left')
# how: left|right|inner|outer
df.merge(other, left_on='a',
    right_on='b')

# default how='inner' — loses rows!
```

## PIVOT & RESHAPE

```
df.pivot_table(index='user',
    columns='month', values='amt',
    aggfunc='sum', fill_value=0)

df.melt(id_vars=['user'],
    var_name='month', value_name='amt')
```

## FILTER ROWS

```
df[df['amt'] > 100]
df[df['col'].isin(['a', 'b'])]
df[df['col'].str.contains('foo',
    na=False)]
df.query("amt > 100 and status=='paid'")
```

## NULL HANDLING

```
df['col'].fillna(0)
df.dropna(subset=['col'])
df['col'].isna()
```

## NEW COLUMN FROM CONDITION

```
df['flag'] = (df['amt'] > 100).astype(int)

df['x'] = np.where(df['a'] > 0,
    df['b'], df['c'])

df['bucket'] = pd.cut(df['amt'],
    bins=[0, 10, 100, 1000],
    labels=['s', 'm', 'l'])
```

## WINDOW-ISH OPS

```
df['cum'] = df.groupby('user')['amt']\
    .cumsum()
df['rk'] = df.groupby('dept')['sal']\
    .rank(method='dense',
    ascending=False)
df['prev'] = df.groupby('user')['amt']\
    .shift(1)
df['roll'] = df['amt'].rolling(7).mean()
```

## DATES

```
df['dt'] = pd.to_datetime(df['dt'])
df['month'] = df['dt'].dt.to_period('M')
df['dow'] = df['dt'].dt.day_name()
df.sort_values('dt')
```

## SORT & INSPECT

```
df.sort_values(['a', 'b'],
    ascending=[True, False])
df['col'].value_counts()
df.describe(); df.info(); df.shape
df.head(); df['col'].unique()
df['col'].nunique()
```

## QUICK REFS

```
# Comprehensions
[f(x) for x in xs if cond]
{k: v for k, v in pairs}
{x for x in xs}

# Strings
s.split(','); ', '.join(parts)
s.strip().lower()
s.startswith('x'); 'x' in s; s[::-1]
s.replace('a', 'b')

# Slices
a[start:stop:step]; a[-3:]; a[::-1]

# Dict / set
d.get(k, default)
d.setdefault(k, [])
a & b (intersect)  a | b (union)
a - b (difference)

# HackerRank stdin
n = int(input())
arr = list(map(int, input().split()))
```

## Pandas / Python Gotchas

- Chained df[df.x>0][y]=v silently fails. Use .loc[mask,y]=v.
- merge default is how='inner'.
- groupby().agg() drops grouping cols unless as\_index=False.
- Mutable default args: def f(a=[]) — don't.
- range(n) is 0..n-1. Off-by-one is interview kryptonite.

# DSA Templates — Patterns to Memorize

Page 3 of 3

The rustiest area for a working analyst. If you recognize the pattern, you've already won.

## HASHMAP COUNTING

```
from collections import Counter, \
    defaultdict

counts = Counter(arr)          # {v: n}
Counter(arr).most_common(k)    # top-K

d = defaultdict(list)
for k, v in pairs:
    d[k].append(v)
```

## TWO-SUM (HASHMAP)

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target-x], i]
        seen[x] = i
```

## TWO POINTERS (SORTED ARRAY)

```
i, j = 0, len(a) - 1
while i < j:
    s = a[i] + a[j]
    if s == target:
        return (i, j)
    elif s < target:
        i += 1
    else:
        j -= 1
```

## SLIDING WINDOW

```
# Longest substring w/o repeating char
def longest(s):
    seen = {}; left = 0; best = 0
    for right, c in enumerate(s):
        if c in seen and seen[c] >= left:
            left = seen[c] + 1
        seen[c] = right
        best = max(best, right-left+1)
    return best
```

## PREFIX SUM

```
# Count subarrays summing to k
def subarray_sum(nums, k):
    seen = {0: 1}
    total = 0; count = 0
    for x in nums:
        total += x
        count += seen.get(total-k, 0)
        seen[total] = seen.get(total, 0) + 1
    return count
```

## SORT WITH KEY

```
arr.sort(key=lambda x: (-x[1], x[0]))
# by 2nd desc, then 1st asc
```

```
sorted(items, key=lambda t: t[0])
```

## GROUP CONSECUTIVE

```
from itertools import groupby
for key, grp in groupby(arr):
    print(key, list(grp))
# arr must be sorted by key first
```

## BINARY SEARCH (TEMPLATE)

```
def bsearch(a, target):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == target: return mid
        if a[mid] < target: lo = mid+1
        else: hi = mid-1
    return -1
```

```
# or: bisect.bisect_left(a, x)
```

## STACK — VALID PARENS

```
def valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for c in s:
        if c in '([{':
            stack.append(c)
        elif (not stack or
              stack.pop() != pairs[c]):
            return False
    return not stack
```

## BFS SKELETON

```
from collections import deque
def bfs(start, graph):
    q = deque([start])
    seen = {start}
    while q:
        node = q.popleft()
        for nb in graph[node]:
            if nb not in seen:
                seen.add(nb)
                q.append(nb)
```

## Last 30 Seconds Before Screen-Share

- Notebook + pen out. Water in reach.
- Editor open, scratch file ready, comment block per problem.
- Mic check — verbal coding is the play.
- Smile. Bank the easy ones first. You've shipped harder than this at work.